

Examen Réparti 1 PSCR Master 1 Informatique Déc 2020

UE 41400

Année 2020-2021

2 heures – **Tout document papier autorisé**
Tout appareil de communication électronique interdit (téléphones...)

Introduction

- Le barème est sur 20 points et est donné à titre indicatif.
- Dans le code C++ demandé vous vous efforcerez d'écrire du code compilable. ✓
- On ne demande dans le code ni les include, ni les qualifications de namespace `std::`.
- On se permettra de fusionner déclarations et implantations, placez le code des opérations directement dans la classe.
- On considère que les appels système n'échouent pas. ✓
- Si on vous demande des modifications sur un code, ne recopiez pas tout, référez aux numéros de lignes.

Le sujet est composé de trois exercices indépendants qu'on pourra traiter dans l'ordre qu'on souhaite.

1 ResultatCalcul (9 points)

1.1 Classe ResultatCalcul

La classe `ResultatCalcul` héberge un résultat (un `std::string` pour s'épargner une version générique) qui doit être calculé par un thread esclave et accédé par un thread maître. Il permet au maître d'attendre la complétion du travail du ou des esclaves.

On propose l'API suivante :

- `const string & getResult() const` : Rend la valeur stockée dès qu'elle est disponible. L'invocation bloque l'appelant si nécessaire jusqu'à ce que la valeur devienne disponible.
- `bool isSet() const` : est vrai si et seulement si la valeur a déjà été affectée, i.e. `getResult` n'est plus bloquant.
- `void setResult(const string & res)` : permet de positionner la valeur stockée, débloque les threads éventuellement bloqués sur `getResult`. On ne doit invoquer `setResult` qu'une seule fois sur une instance donnée de `ResultatCalcul`, si on l'invoque de nouveau cela n'a aucun effet.

Question 1. (3 points) Planter cette classe avec son constructeur par défaut et si nécessaire un destructeur.

1.2 Utilisation de la classe

On souhaite écrire un programme qui :

- crée un vecteur de `ResultatCalcul` de taille N ✓
- construit N threads esclave, chaque esclave va calculer un des résultats : la valeur calculée sera simplement "slaveXX" où XX est son *numéro d'ordre de création*.
- affiche le contenu du vecteur, une valeur par ligne, quand il est calculé ←
- se termine proprement.

Question 2. (2 points) Proposez un programme C++ (un `main`) qui a ce comportement et qui maximise le parallélisme. On interdit l'utilisation de variables globales. ✓

Quand N est grand, on constate que créer trop de threads n'est pas rentable. On souhaite au contraire ne construire que K threads esclave (au lieu de N), en supposant K petit devant N . Chaque esclave dans cette version doit traiter N/K tâches (on ignore l'arrondi, N est multiple de K).

On fait donc un découpage par lots de taille égale des résultats à calculer, chaque lot ayant une taille N/K .

Question 3. (2 points) Modifiez votre programme pour avoir ce comportement.

1.3 Comparaison à un Pool

On voudrait comparer le comportement de votre programme à une version qui utiliserait un pool de threads, comme élaboré en TD/TME, et construirait un Job pour chaque résultat à calculer. (NB : on ne demande pas le code de cette version !)

Question 4. (1,5 points) Expliquez en quoi la solution avec un Pool diffère de la solution proposée en Q3 si on suppose que le calcul de chaque résultat a une durée fixe (homogène) ou au contraire que le calcul d'un résultat prend une durée aléatoire difficile à prédire. On s'intéresse à l'utilisation des ressources de la machine en temps total de calcul sur une machine multi-core.

2 Arbre, pointeurs, gestion mémoire en C++ (5 points)

On considère un arbre binaire de recherche, chaque noeud de l'arbre (on utilisera une seule classe) porte deux pointeurs vers les sous-arbres gauche et droit (qui peuvent être `nullptr` désignant un sous-arbre vide), ainsi qu'une string. Le sous-arbre gauche (resp. droit) contient des string lexico-graphiquement inférieures strictement (respectivement supérieures strictement) à la string du noeud courant. On a un ensemble au sens mathématique, un seul noeud de l'arbre porte une string donnée.

Question 1. Donnez le code de la classe `Node` :

- (0,5 pt) ses attributs `left`, `right` et `value`.
- (0,5 pt) un constructeur à un seul argument (la string) qui initialise les sous arbres gauche et droit à vide.
- (2 pt) `bool insert(const string & s)` : essaie d'insérer la string s et rend `yrai` si une insertion a lieu, et `faux` sinon. Soit le noeud courant porte déjà la string à insérer, soit il faut réaliser une récursion sur le sous arbre adapté (en comparant s à la valeur stockée). Si la récursion atteint un sous arbre vide on crée un nouveau noeud qu'on accroche dans l'arbre.
- (1 pt) un constructeur par copie, qui suit la signature standard `Node(const Node &)` et copie tout l'arbre. On ne demande pas l'opérateur d'affectation qui serait similaire.
- (1 pt) un destructeur qui doit désallouer tout l'arbre dont le noeud courant est la racine.

3 Arbres de processus (6 points)

On considère le code suivant.

fork.cpp

```

1 #include <iostream>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 // a faire varier
6 #define N 4
7
8 int main () {
9     int i = -1;
10    int j = -1;
11    for (i=0; i < N; ++i) {
12        if (fork() == 0) {
13            for (j=i+1; j < N; ++j) {
14                if (fork() != 0) {
15                    break;
16                }
17            }
18            break;
19        }
20    }
21    std::cout << "Processus " << getpid() << std::endl;
22    return 0;
23 }

```

Question 1. (2 points) Donnez le nombre total (sans compter le main) de processus engendrés pour $N=1$, $N=2$, $N=3$, $N=4$.

Question 2. (1 point) Exprimez le nombre processus engendrés comme une fonction de N .

Question 3. (1,5 points) Tout en maximisant le parallélisme, ajouter les instructions `wait` utiles juste avant le `return 0`; pour garantir que le main ne se termine qu'après que l'ensemble de ses descendants soient terminés. NB: On évitera d'invoquer `wait` plus de fois que l'on a créé de processus fils. On recommande de s'appuyer sur les valeurs de i et j pour décider du comportement à adopter après les boucles, ou d'introduire des nouvelles variables au choix.

Question 4. (1,5 points) Les instructions de synchronisation `wait` sont placées actuellement juste après l'affichage; on décide plutôt de placer ces instruction avant l'affichage sur `std::cout`. Quelles sont les garanties sur l'ordre dans lequel les affichages seront produits dans ces deux solutions ?